

OLED + Integration

UMD CS | Spring 26



Digital Input/Output

Analog Input/Output

Motor / Ultrasound

IMU

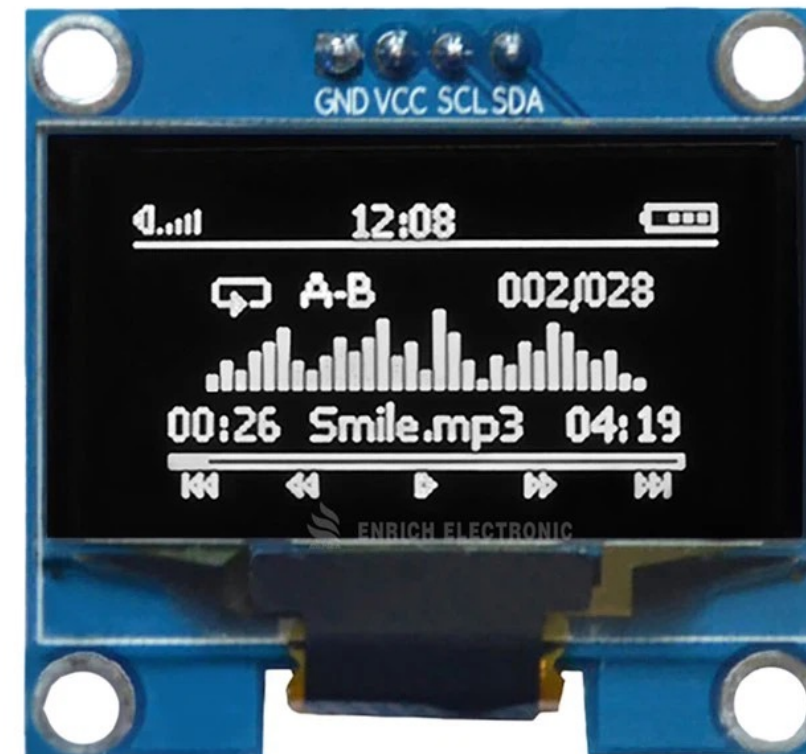
+

Shift Register

3D Modeling

Laser Cutting

...



0.49 Inch OLED Display Module

128×64

I2C Interface

4 Pin

3.3-5V

SSD1306



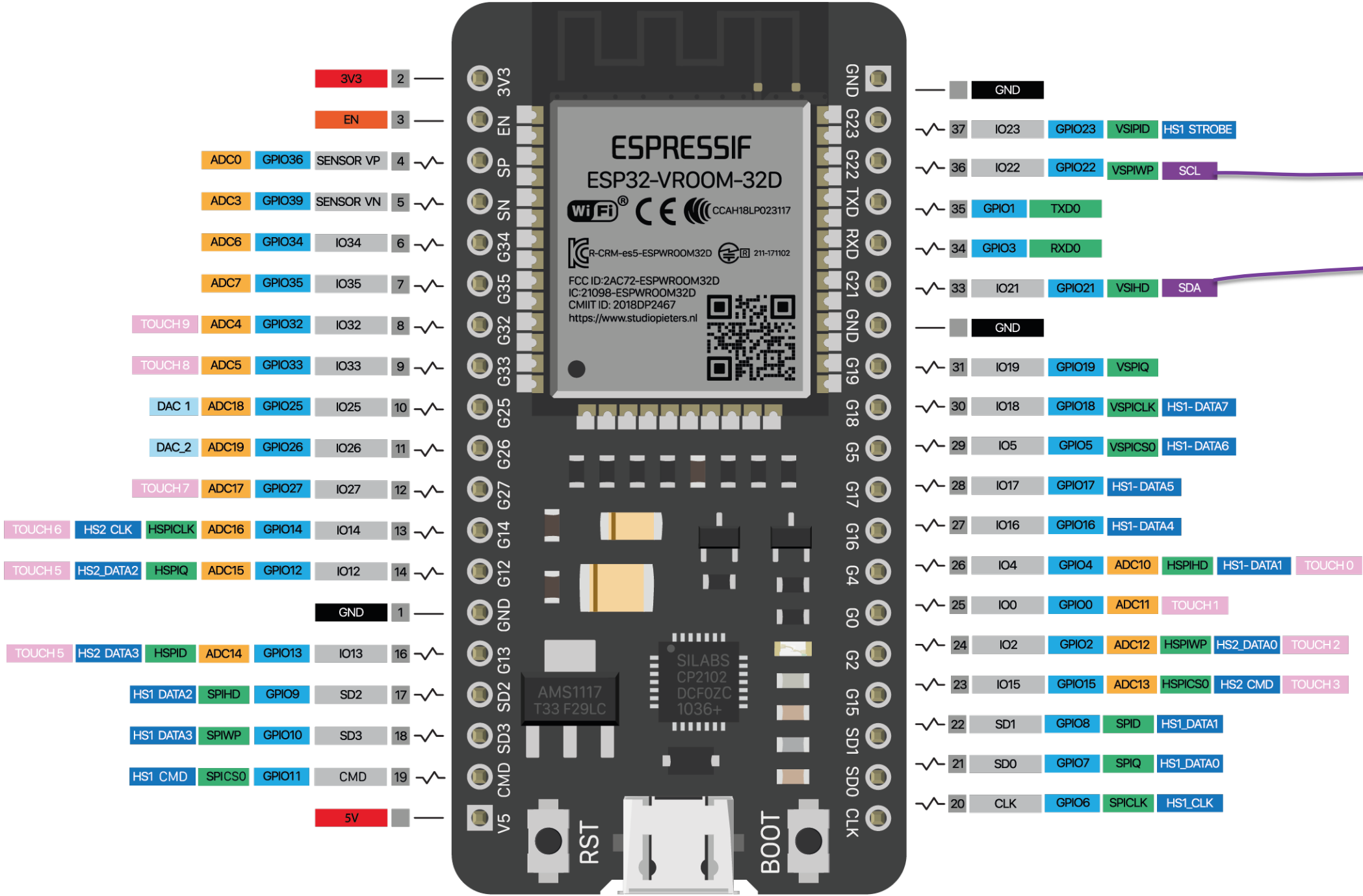
OLED - a display technology where each pixel is made of organic compounds that emit light when an electric current is applied. It provides high contrast, vibrant colors, and doesn't require a backlight, making it ideal for thin, flexible displays. It doesn't use backlight to illuminate the display (as in LCDs with LED backlighting).

SSD1306 is a single chip CMOS OLED/PLED driver with controller for OLED dot-matrix graphic display. It supports several communication mode, I2C, SPI, and Parallel Interface.

To establish I2C communication between the master and slave devices, only two channels are needed: SDA and CLK.

5 min finish the wiring?

- PWM
- PIN NUMBER
- NAME
- GROUND
- POWER
- CONTROL
- I/O
- ADC
- COMM. INTERFACE
- DAC
- I2C
- HS
- TOUCH



SSD1306

Advance Information

128 x 64 Dot Matrix
OLED/PLED Segment/Common Driver with Controller

This document contains information on a new product. Specifications and information herein are subject to change without notice.



CONTENTS

1	GENERAL DESCRIPTION	6
2	FEATURES.....	6
3	ORDERING INFORMATION	6
4	BLOCK DIAGRAM	7
5	DIE PAD FLOOR PLAN	8
6	PIN ARRANGEMENT	11
6.1	SSD1306TR1 PIN ASSIGNMENT	11
7	PIN DESCRIPTION	13
8	FUNCTIONAL BLOCK DESCRIPTIONS.....	15
8.1	MCU INTERFACE SELECTION.....	15
8.1.1	MCU Parallel 6800-series Interface.....	15
8.1.2	MCU Parallel 8080-series Interface.....	16
8.1.3	MCU Serial Interface (4-wire SPI).....	17
8.1.4	MCU Serial Interface (3-wire SPI).....	18
8.1.5	MCU I ² C Interface.....	19
8.2	COMMAND DECODER	22
8.3	OSCILLATOR CIRCUIT AND DISPLAY TIME GENERATOR.....	22
8.4	FR SYNCHRONIZATION	23
8.5	RESET CIRCUIT	23
8.6	SEGMENT DRIVERS / COMMON DRIVERS	24
8.7	GRAPHIC DISPLAY DATA RAM (GDDRAM).....	25
8.8	SEG/COM DRIVING BLOCK	26
8.9	POWER ON AND OFF SEQUENCE	27
9	COMMAND TABLE	28
9.1	DATA READ / WRITE	33
10	COMMAND DESCRIPTIONS	34
10.1	FUNDAMENTAL COMMAND	34
10.1.1	Set Lower Column Start Address for Page Addressing Mode (00h~0Fh)	34
10.1.2	Set Higher Column Start Address for Page Addressing Mode (10h~1Fh)	34
10.1.3	Set Memory Addressing Mode (20h).....	34
10.1.4	Set Column Address (21h)	35
10.1.5	Set Page Address (22h).....	36
10.1.6	Set Display Start Line (40h~7Fh)	36
10.1.7	Set Contrast Control for BANK0 (81h).....	36
10.1.8	Set Segment Re-map (A0h/A1h).....	36
10.1.9	Entire Display ON (A4h/A5h)	37
10.1.10	Set Normal/Inverse Display (A6h/A7h).....	37
10.1.11	Set Multiplex Ratio (A8h).....	37
10.1.12	Set Display ON/OFF (AEh/AFh)	37
10.1.13	Set Page Start Address for Page Addressing Mode (B0h~B7h).....	37
10.1.14	Set COM Output Scan Direction (C0h/C8h).....	37
10.1.15	Set Display Offset (D3h)	37
10.1.16	Set Display Clock Divide Ratio/ Oscillator Frequency (D5h)	40
10.1.17	Set Pre-charge Period (D9h).....	40
10.1.18	Set COM Pins Hardware Configuration (DAh).....	40
10.1.19	Set V _{COMH} Deselect Level (DBh)	43

7 PIN DESCRIPTION

Key:

I = Input	NC = Not Connected
O =Output	Pull LOW= connect to Ground
I/O = Bi-directional (input/output)	Pull HIGH= connect to V _{DD}
P = Power pin	

Figure 7-1 Pin Description

Pin Name	Type	Description
V _{DD}	P	Power supply pin for core logic operation.
V _{CC}	P	Power supply for panel driving voltage. This is also the most positive power voltage supply pin.
V _{SS}	P	This is a ground pin.
V _{LSS}	P	This is an analog ground pin. It should be connected to V _{SS} externally.
V _{COMH}	O	The pin for COM signal deselected voltage level. A capacitor should be connected between this pin and V _{SS} .
V _{BAT}	P	Reserved pin. It should be connected to V _{DD} .
BGGND	P	Reserved pin. It should be connected to ground.
C1P/C1N C2P/C2N	I	Reserved pin. It should be kept NC.
V _{BREF}	P	Reserved pin. It should be kept NC.
BS[2:0]	I	MCU bus interface selection pins. Please refer to Table 7-1 for the details of setting.
I _{REF}	I	This is segment output current reference pin. A resistor should be connected between this pin and V _{SS} to maintain the I _{REF} current at 12.5 uA. Please refer to Figure 8-15 for the details of resistor value.
FR	O	This pin outputs RAM write synchronization signal. Proper timing between MCU data writing and frame display timing can be achieved to prevent tearing effect. It should be kept NC if it is not used. Please refer to Section 8.4 for details usage.
CL	I	This is external clock input pin. When internal clock is enabled (i.e. HIGH in CLS pin), this pin is not used and should be connected to V _{SS} . When internal clock is disabled (i.e. LOW in CLS pin), this pin is the external clock source input pin.
CLS	I	This is internal clock enable pin. When it is pulled HIGH (i.e. connect to V _{DD}), internal clock is enabled. When it is pulled LOW, the internal clock is disabled; an external clock source must be connected to the CL pin for normal operation.
RES#	I	This pin is reset signal input. When the pin is pulled LOW, initialization of the chip is executed. Keep this pin HIGH (i.e. connect to V _{DD}) during normal operation.
CS#	I	This pin is the chip select input. (active LOW).

Everything we need to know: SSD1306 Datasheet;)

Advance Information

128 x 64 Dot Matrix
OLED/PLED Segment/Common Driver with Controller

This document contains information on a new product. Specifications and information herein are subject to change without notice.



7 PIN DESCRIPTION13

8 FUNCTIONAL BLOCK DESCRIPTIONS.....15

8.1 MCU INTERFACE SELECTION.....15

8.1.1 MCU Parallel 6800-series Interface.....15

8.1.2 MCU Parallel 8080-series Interface.....16

8.1.3 MCU Serial Interface (4-wire SPI).....17

8.1.4 MCU Serial Interface (3-wire SPI).....18

8.1.5 MCU I²C Interface.....19

8.2 COMMAND DECODER.....22

8.3 OSCILLATOR CIRCUIT AND DISPLAY TIME GENERATOR.....22

8.4 FR SYNCHRONIZATION.....23

8.5 RESET CIRCUIT.....23

8.6 SEGMENT DRIVERS / COMMON DRIVERS.....24

8.7 GRAPHIC DISPLAY DATA RAM (GDDRAM).....25

8.8 SEG/COM DRIVING BLOCK.....26

8.9 POWER ON AND OFF SEQUENCE.....27

9 COMMAND TABLE28

9.1 DATA READ / WRITE33

10 COMMAND DESCRIPTIONS34

10.1 FUNDAMENTAL COMMAND34

10.1.1 Set Lower Column Start Address for Page Addressing Mode (00h~0Fh).....34

10.1.2 Set Higher Column Start Address for Page Addressing Mode (10h~1Fh).....34

10.1.3 Set Memory Addressing Mode (20h).....34

10.1.4 Set Column Address (21h).....35

10.1.5 Set Page Address (22h).....36

10.1.6 Set Display Start Line (40h~7Fh).....36

10.1.7 Set Contrast Control for BANK0 (81h).....36

10.1.8 Set Segment Re-map (A0h/A1h).....36

10.1.9 Entire Display ON (A4h/A5h).....37

10.1.10 Set Normal/Inverse Display (A6h/A7h).....37

10.1.11 Set Multiplex Ratio (A8h).....37

10.1.12 Set Display ON/OFF (AEh/AFh).....37

10.1.13 Set Page Start Address for Page Addressing Mode (B0h~B7h).....37

10.1.14 Set COM Output Scan Direction (C0h/C8h).....37

10.1.15 Set Display Offset (D3h).....37

10.1.16 Set Display Clock Divide Ratio/ Oscillator Frequency (D5h).....40

10.1.17 Set Pre-charge Period (D9h).....40

10.1.18 Set COM Pins Hardware Configuration (DAh).....40

10.1.19 Set V_{COMH} Deselect Level (DBh).....43

V _{SS}	P	This is a ground pin.
V _{LSS}	P	This is an analog ground pin. It should be connected to V _{SS} externally.
V _{COMH}	O	The pin for COM signal deselected voltage level. A capacitor should be connected between this pin and V _{SS} .
V _{BAT}	P	Reserved pin. It should be connected to V _{DD} .
BGGND	P	Reserved pin. It should be connected to ground.
C1P/C1N C2P/C2N	I	Reserved pin. It should be kept NC.
V _{BREF}	P	Reserved pin. It should be kept NC.
BS[2:0]	I	MCU bus interface selection pins. Please refer to Table 7-1 for the details of setting.
I _{REF}	I	This is segment output current reference pin. A resistor should be connected between this pin and V _{SS} to maintain the I _{REF} current at 12.5 uA. Please refer to Figure 8-15 for the details of resistor value.
FR	O	This pin outputs RAM write synchronization signal. Proper timing between MCU data writing and frame display timing can be achieved to prevent tearing effect. It should be kept NC if it is not used. Please refer to Section 8.4 for details usage.
CL	I	This is external clock input pin. When internal clock is enabled (i.e. HIGH in CLS pin), this pin is not used and should be connected to V _{SS} . When internal clock is disabled (i.e. LOW in CLS pin), this pin is the external clock source input pin.
CLS	I	This is internal clock enable pin. When it is pulled HIGH (i.e. connect to V _{DD}), internal clock is enabled. When it is pulled LOW, the internal clock is disabled; an external clock source must be connected to the CL pin for normal operation.
RES#	I	This pin is reset signal input. When the pin is pulled LOW, initialization of the chip is executed. Keep this pin HIGH (i.e. connect to V _{DD}) during normal operation.
CS#	I	This pin is the chip select input. (active LOW).

Everything we need to know: SSD1306 Datasheet;

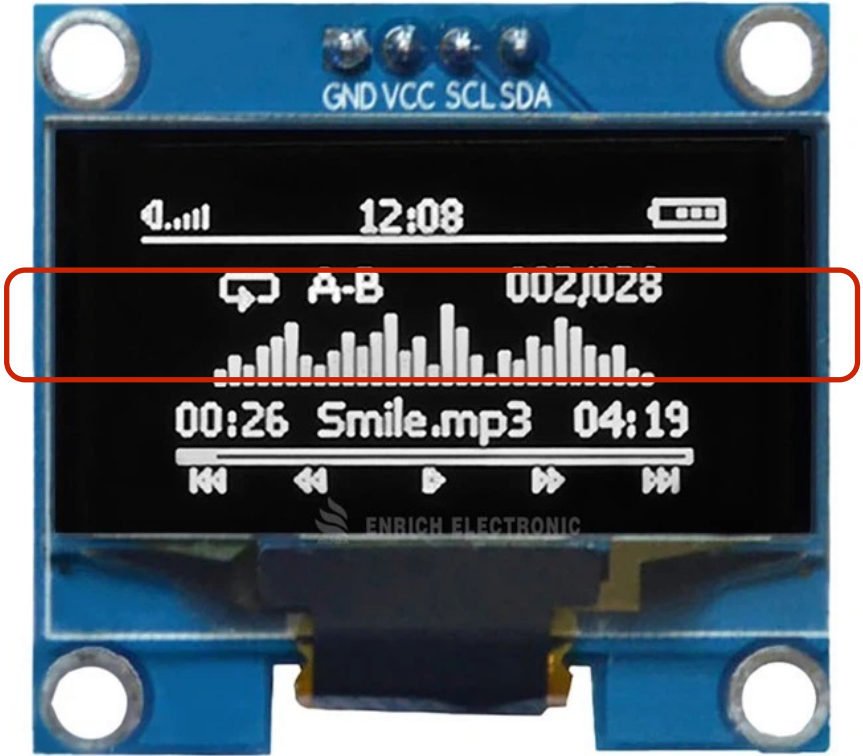
OLED Display

DATASHEET

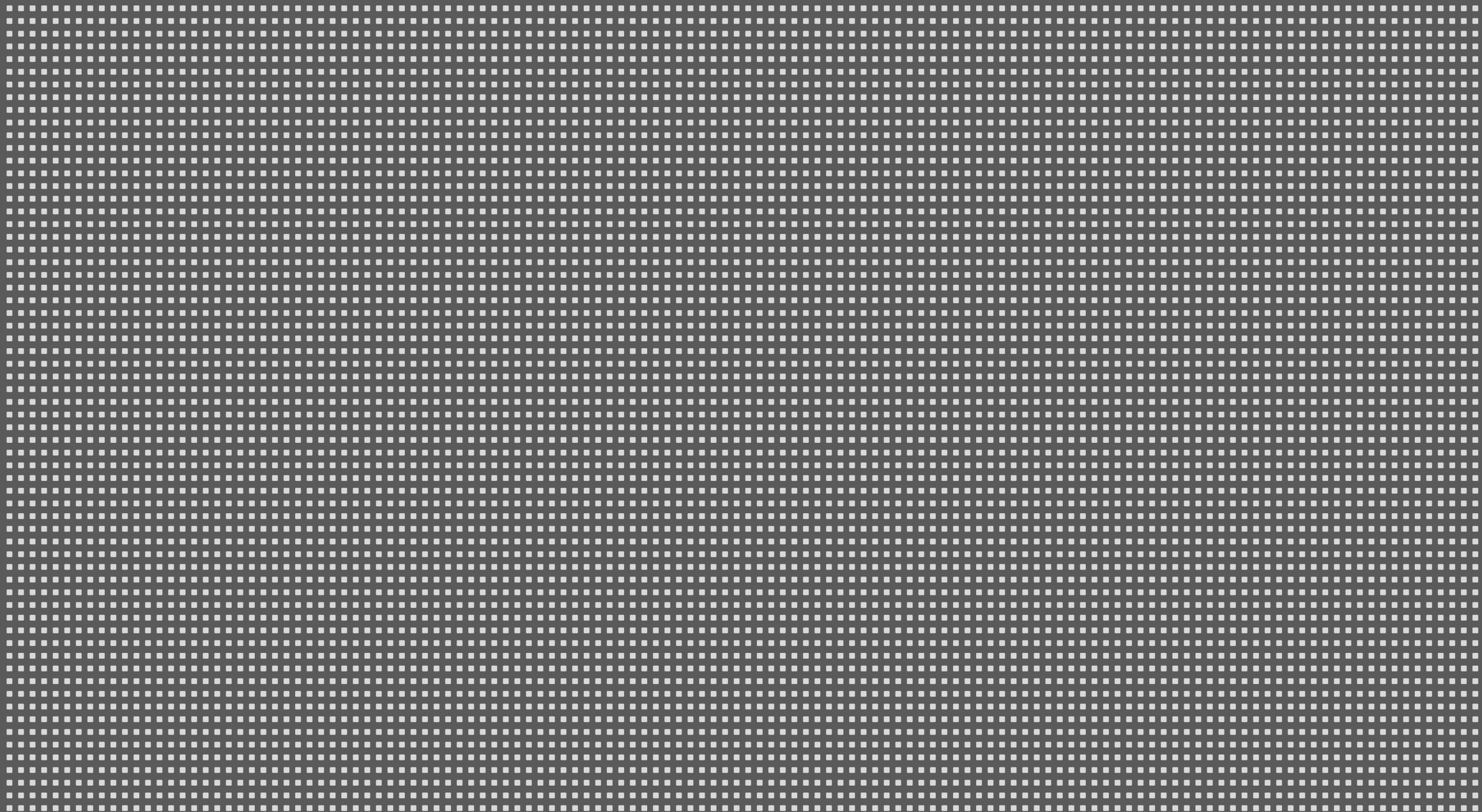
EXAMPLE

https://smartlab.cs.umd.edu/CMSC730/assets/file/SSD1306.pdf

Understanding how to update one pixel of the 128 x 64 array

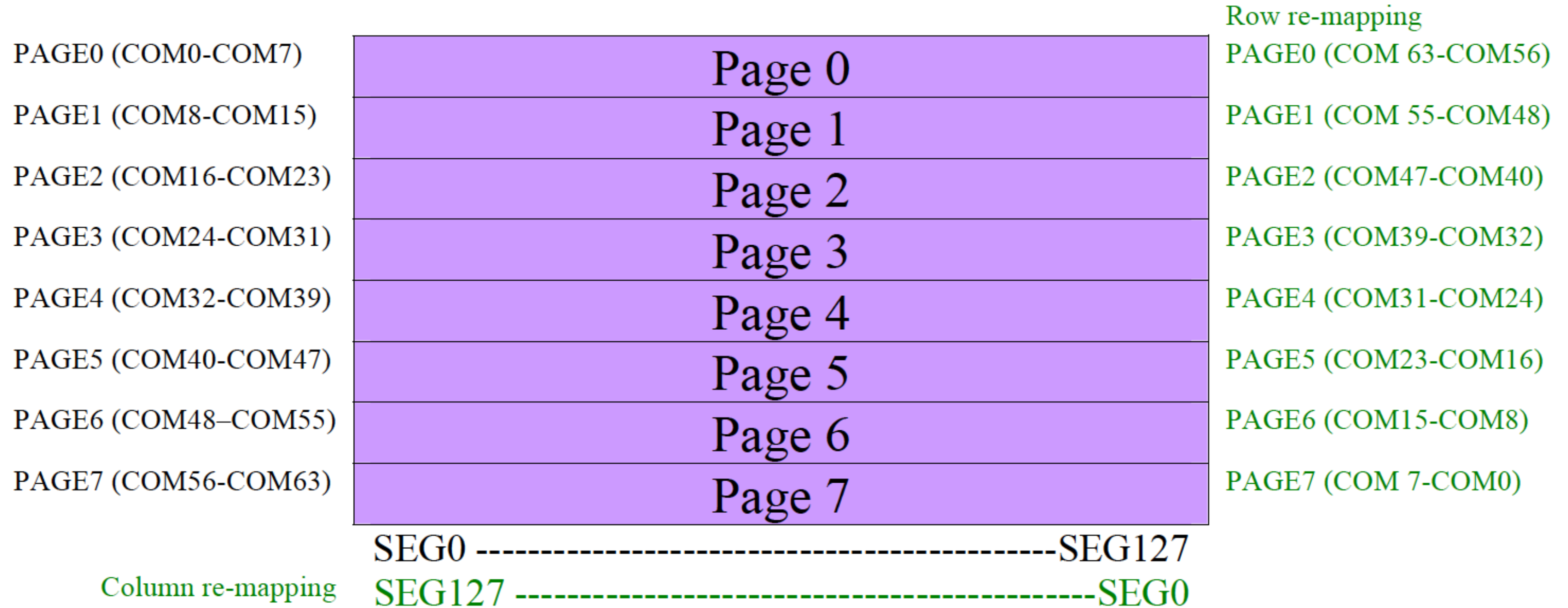


64

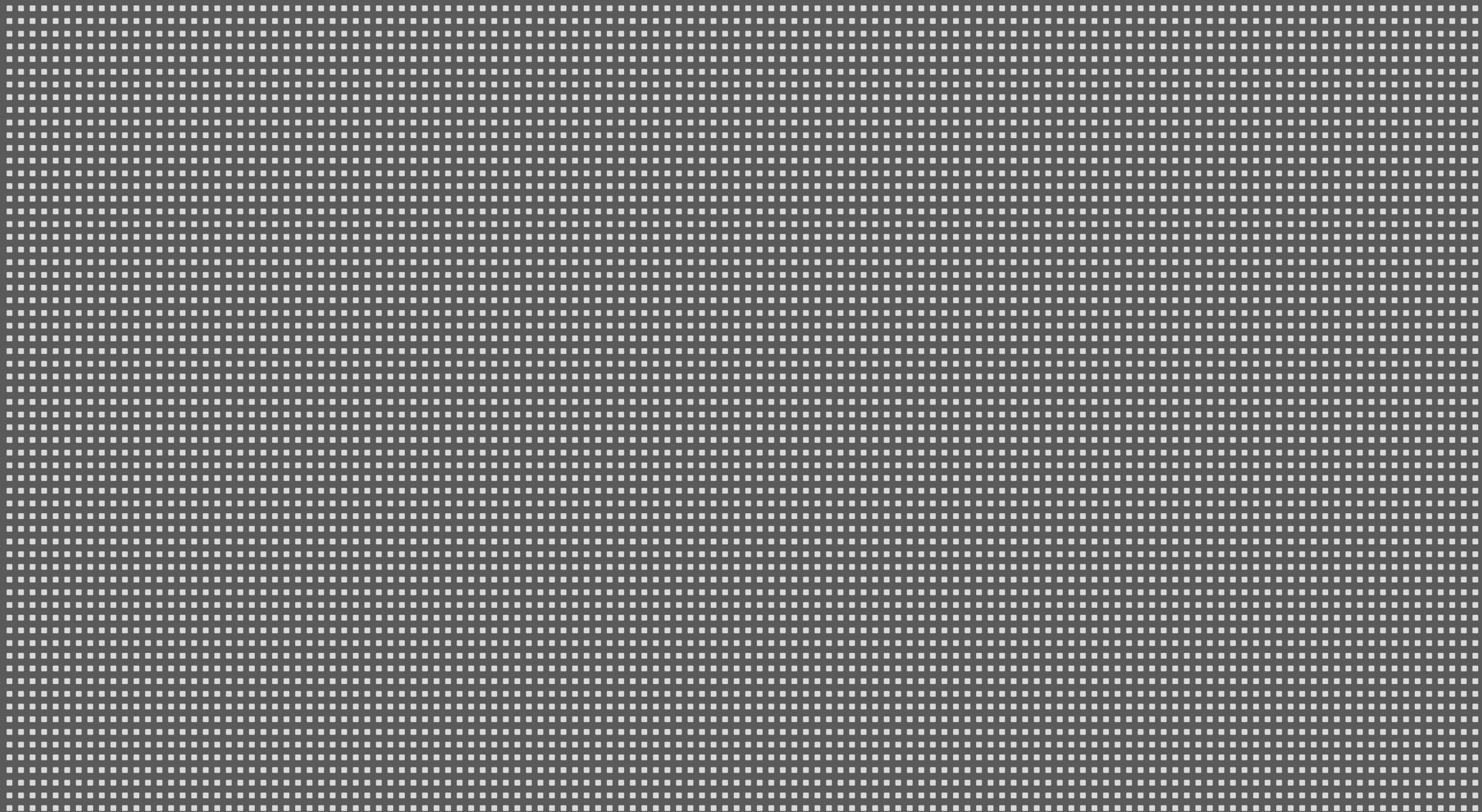


128

Figure 8-13 : GDDRAM pages structure of SSD1306

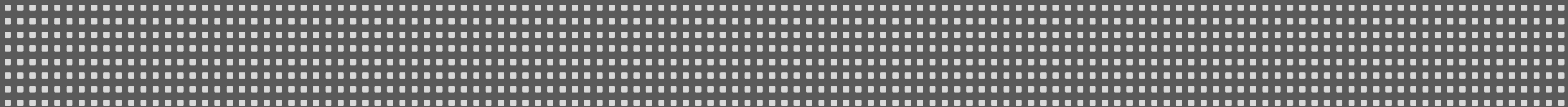


64

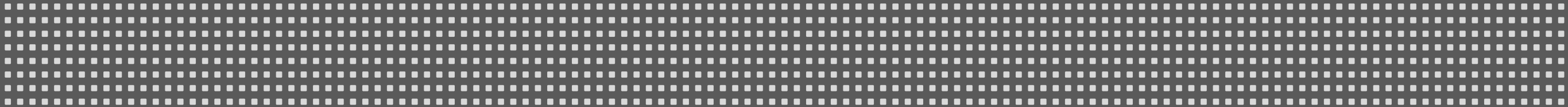


128

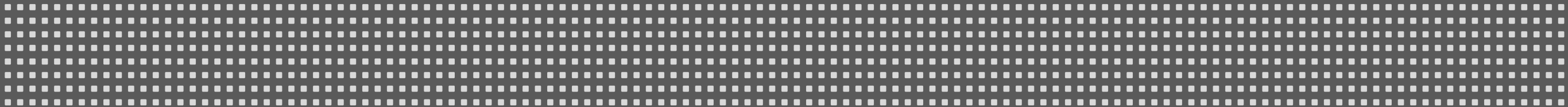
Page 0



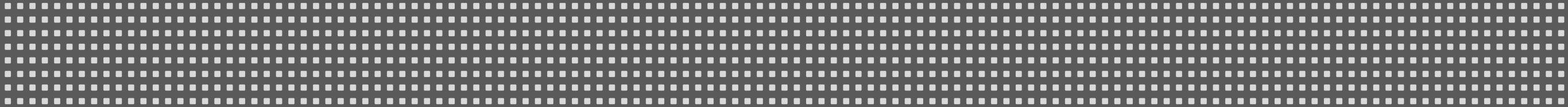
Page 1



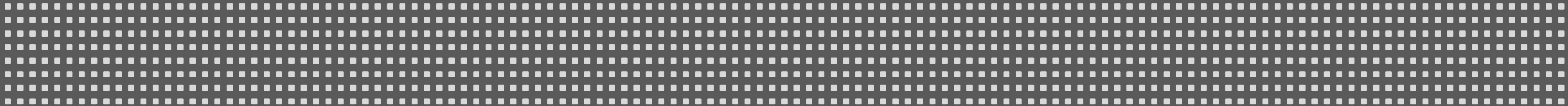
Page 2



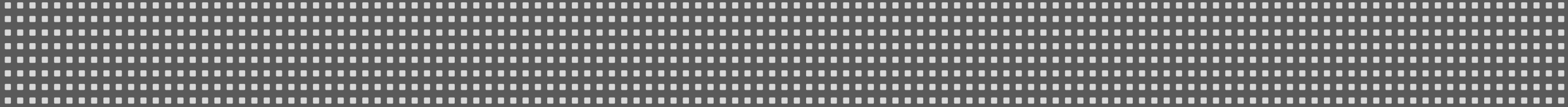
Page 3



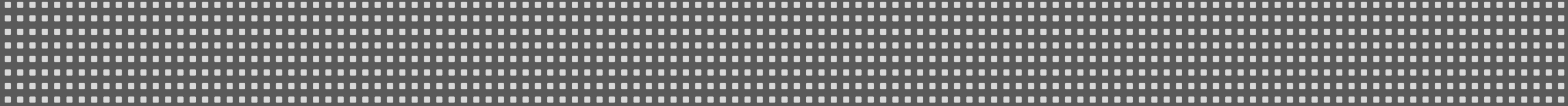
Page 4



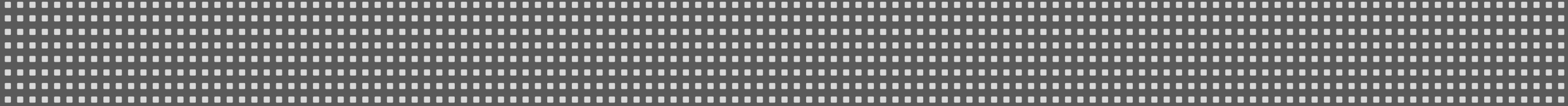
Page 5



Page 6



Page 7



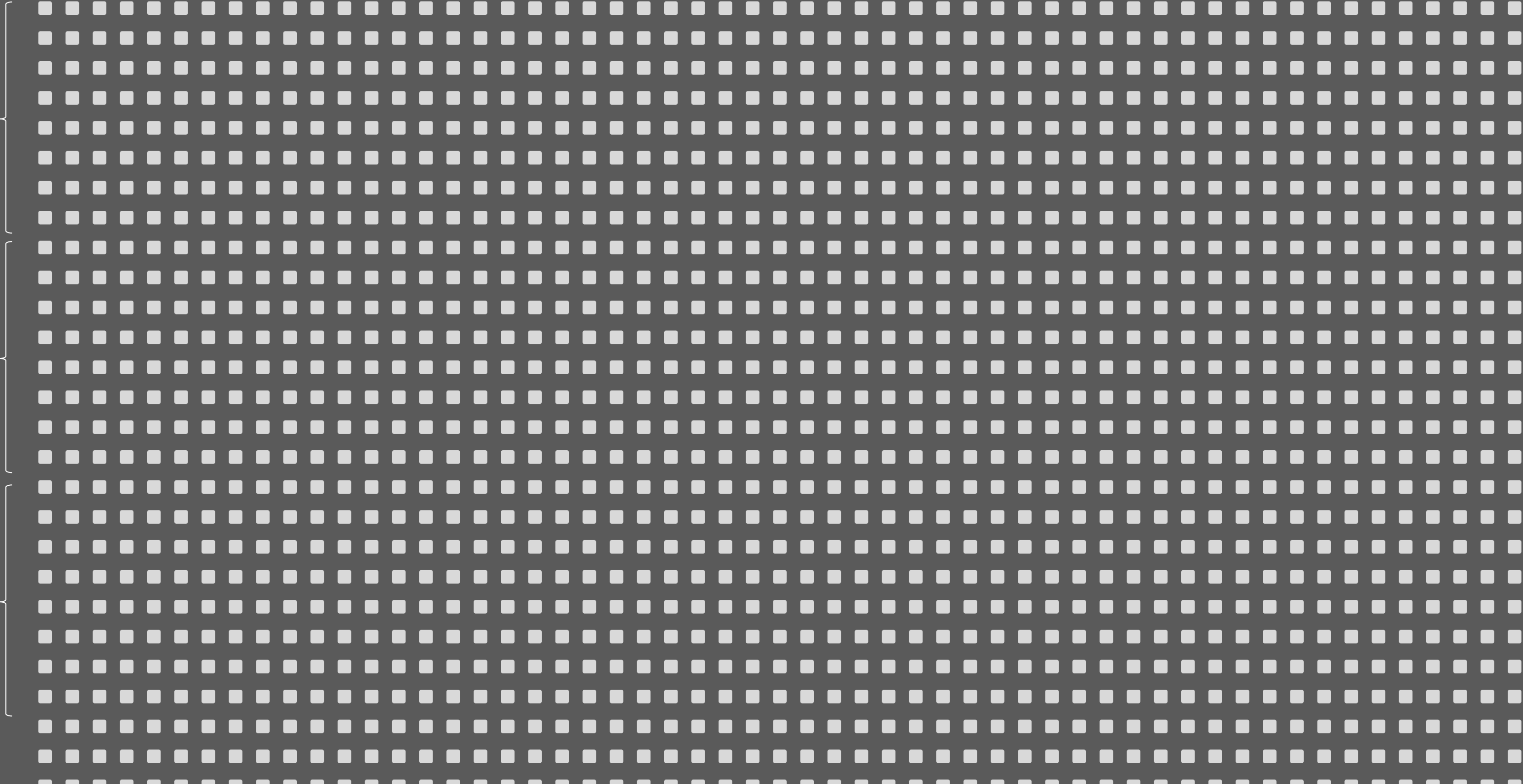
128

Page 0

Page 1

Page 2

⋮



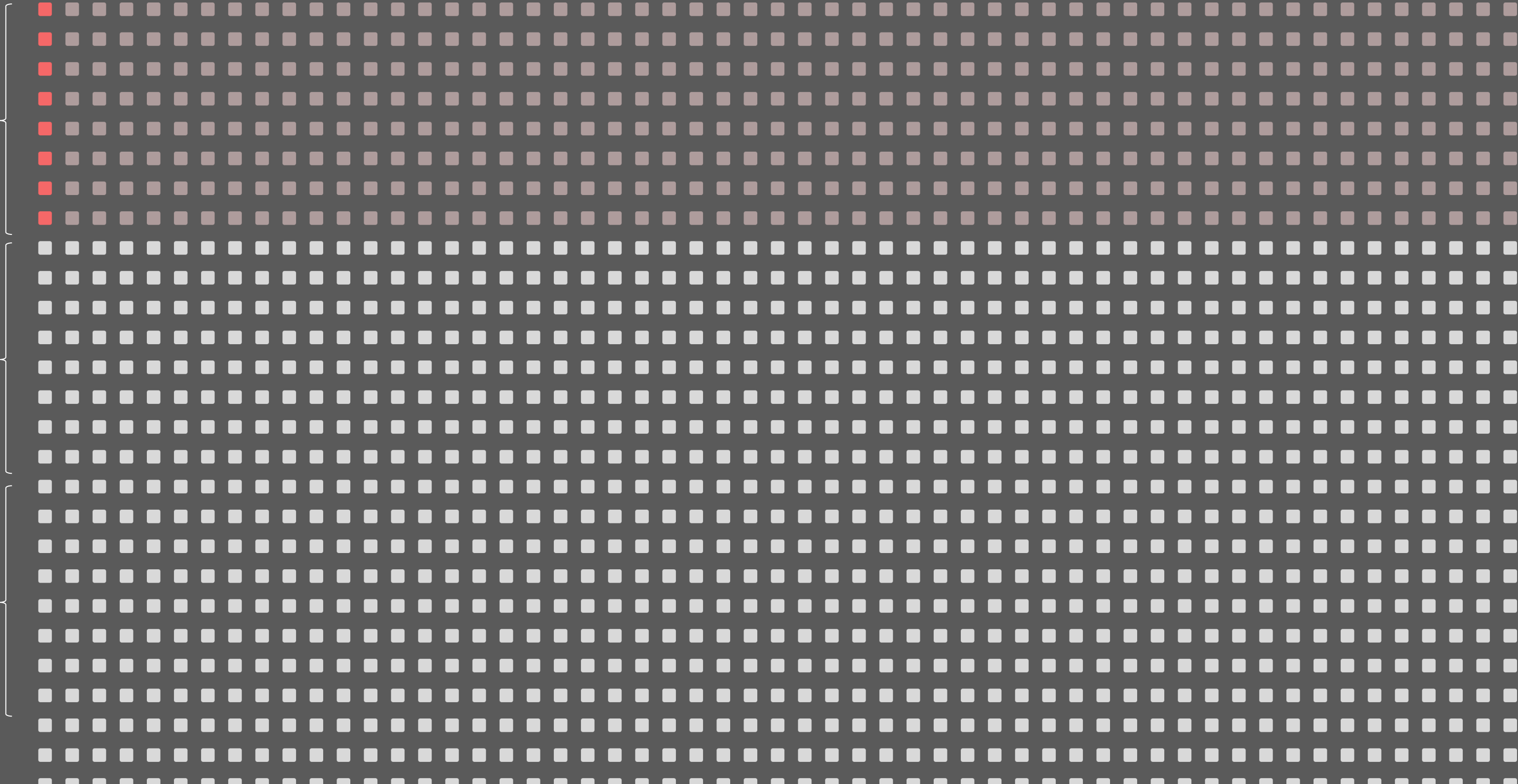
1 byte – 8 bit of data



Page 0

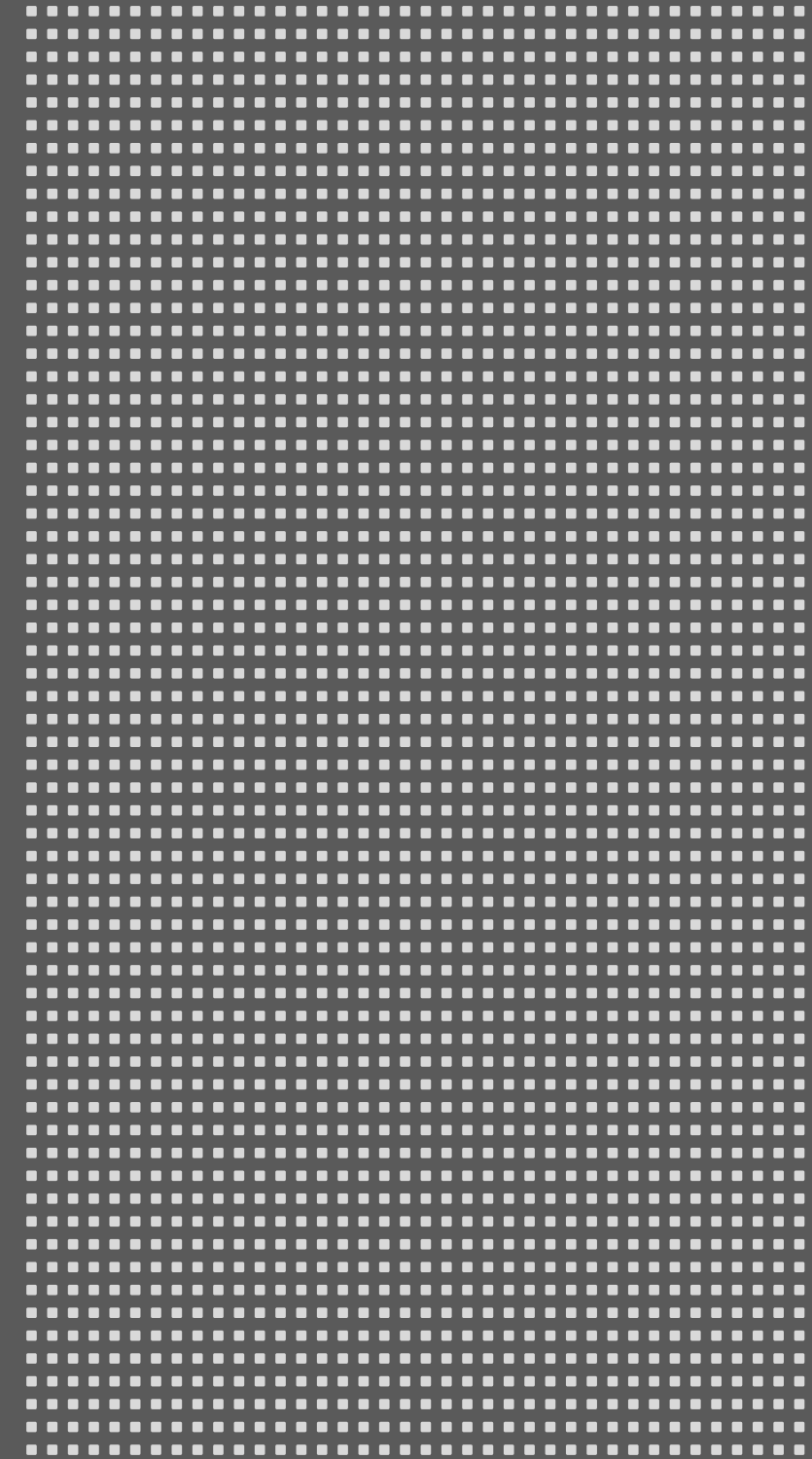
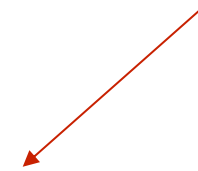
Page 1

Page 2



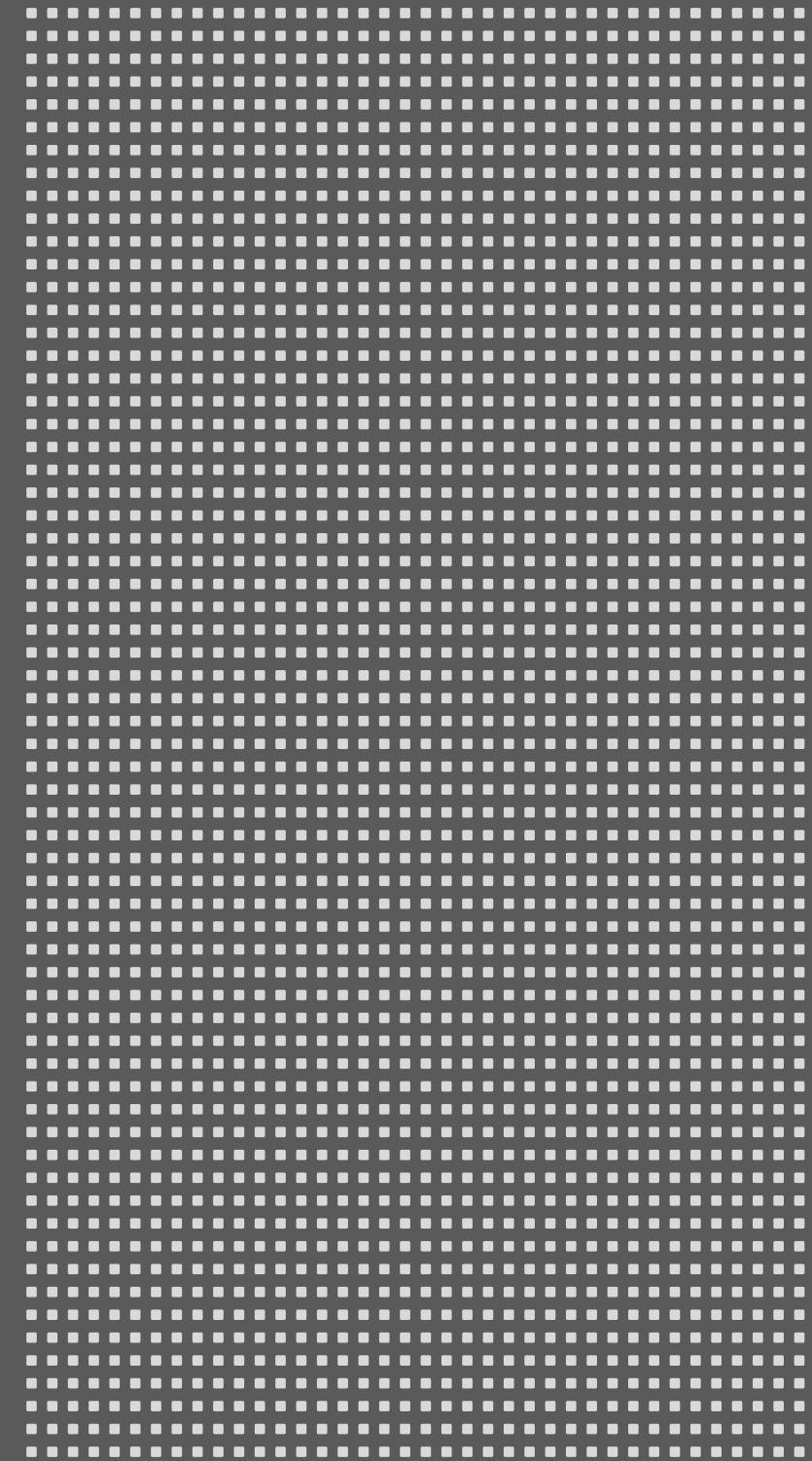
```
#define SCREEN_WIDTH 128  
#define SCREEN_HEIGHT 64  
#define BUFFER_SIZE (SCREEN_WIDTH * SCREEN_HEIGHT / 8)
```

```
uint8_t buffer[BUFFER_SIZE]; // Screen buffer
```




```
#define SCREEN_WIDTH 128  
#define SCREEN_HEIGHT 64  
#define BUFFER_SIZE (SCREEN_WIDTH * SCREEN_HEIGHT / 8)
```

```
uint8_t buffer[BUFFER_SIZE]; // Screen buffer
```



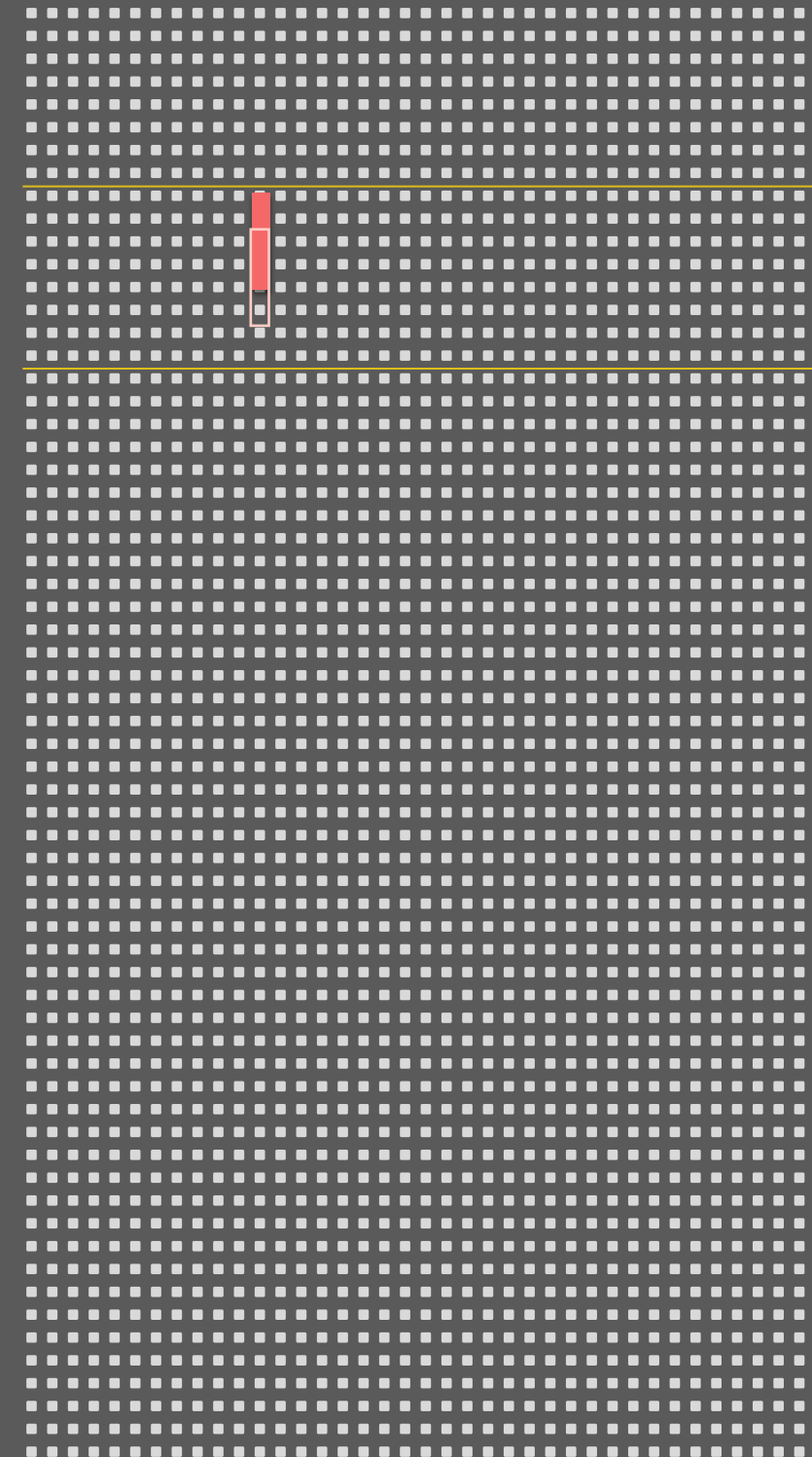
```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define BUFFER_SIZE (SCREEN_WIDTH * SCREEN_HEIGHT / 8)
```

```
uint8_t buffer[BUFFER_SIZE]; // Screen buffer
```

```
void drawPixel(uint8_t x, uint8_t y) {
    uint16_t index = (y / 8) * SCREEN_WIDTH + x;
    buffer[index] |= (1 << (y % 8));
}
```

- 1) Calculate the position of the byte in the buffer [] array:
Which page? $10 / 8 = 1$ (Second page, numbering starts from 0)
So where is it within the buffer? Index = $(10 / 8) * 128 + 11$
- 2) Set the bit within the byte that corresponding to the pixel
Location of the pixel within the byte: $10 \% 8 = 2$
Bitwise shift $<<$, mask the right bit to be 1 : $1 << (10 \% 8)$
OR with the rest without affecting the other bits

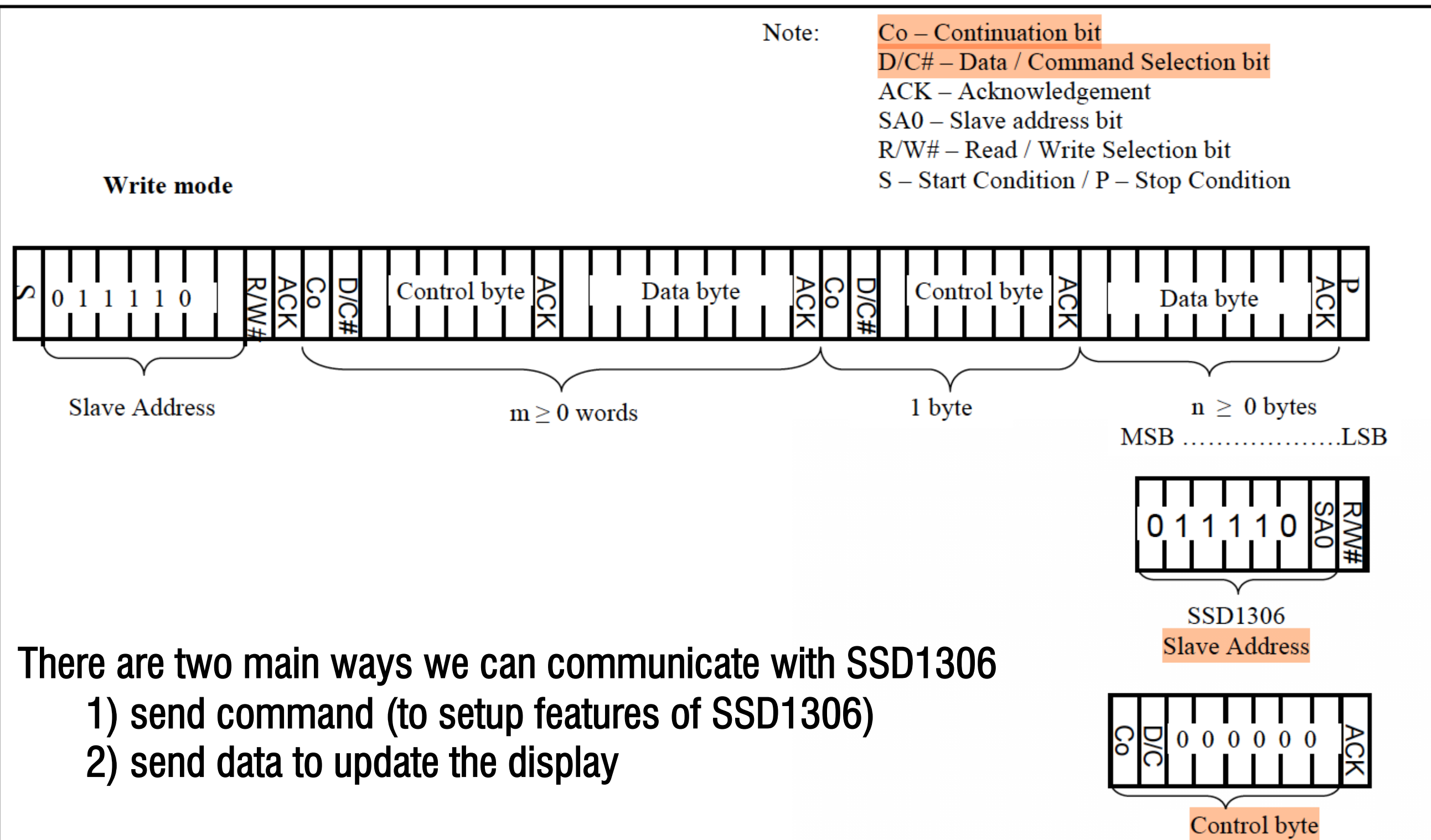
(X,Y) = (11,10)



How to update it to SSD1306? -> Copy the buffer to SSD1306's GDDRAM

How to update it to SSD1306? -> Copy the buffer to SSD1306's GDDRAM

Figure 8-7 : I²C-bus data format



There are two main ways we can communicate with SSD1306

- 1) send command (to setup features of SSD1306)
- 2) send data to update the display

- 5) After the transmission of the slave address, either the control byte or the data byte may be sent across the SDA. A control byte mainly consists of Co and D/C# bits followed by six “0” ‘s.
- a. If the Co bit is set as logic “0”, the transmission of the following information will contain data bytes only.
 - b. The D/C# bit determines the next data byte is acted as a command or a data. If the D/C# bit is set to logic “0”, it defines the following data byte as a command. If the D/C# bit is set to logic “1”, it defines the following data byte as a data which will be stored at the GDDRAM. The GDDRAM column address pointer will be increased by one automatically after each data write.

Note:

Co – Continuation bit

D/C# – Data / Command Selection bit

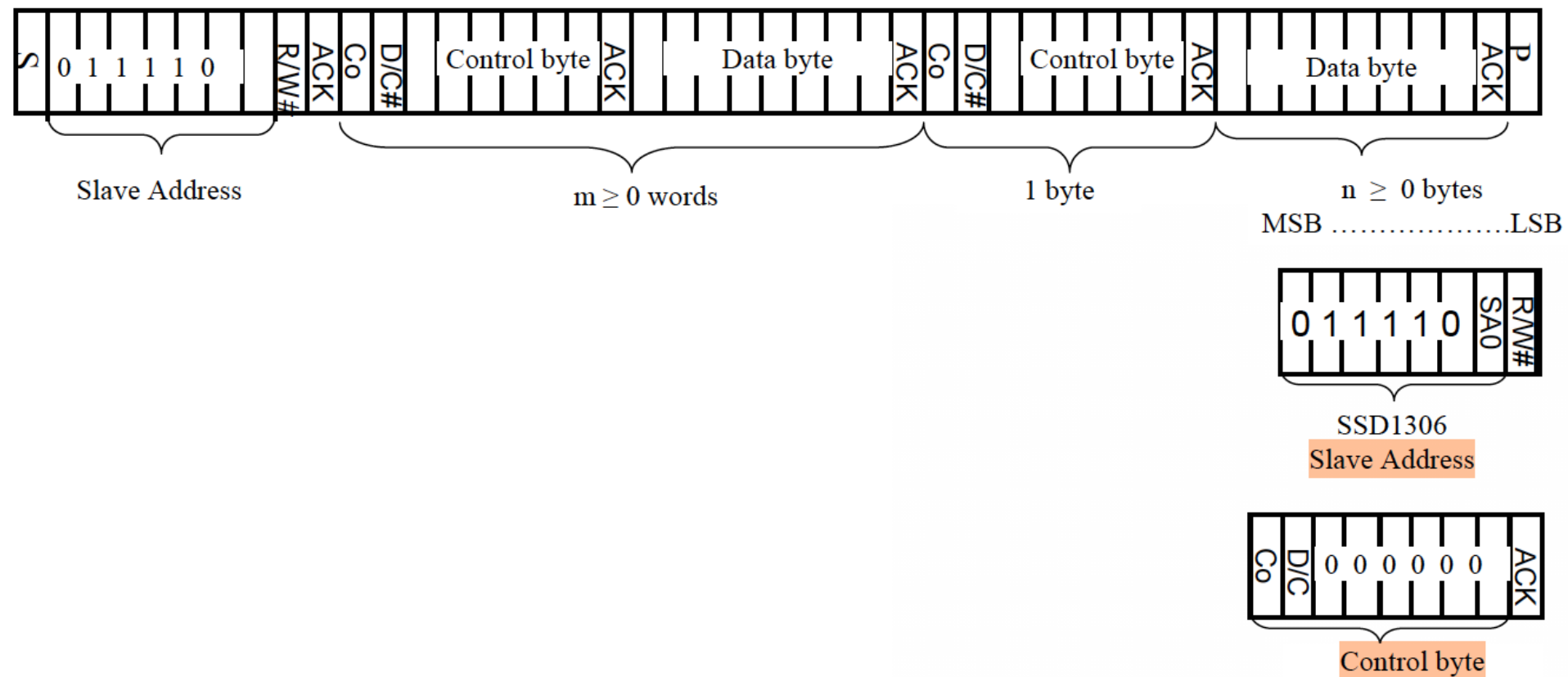
ACK – Acknowledgement

SA0 – Slave address bit

R/W# – Read / Write Selection bit

S – Start Condition / P – Stop Condition

Write mode



Control byte: 0b00000000

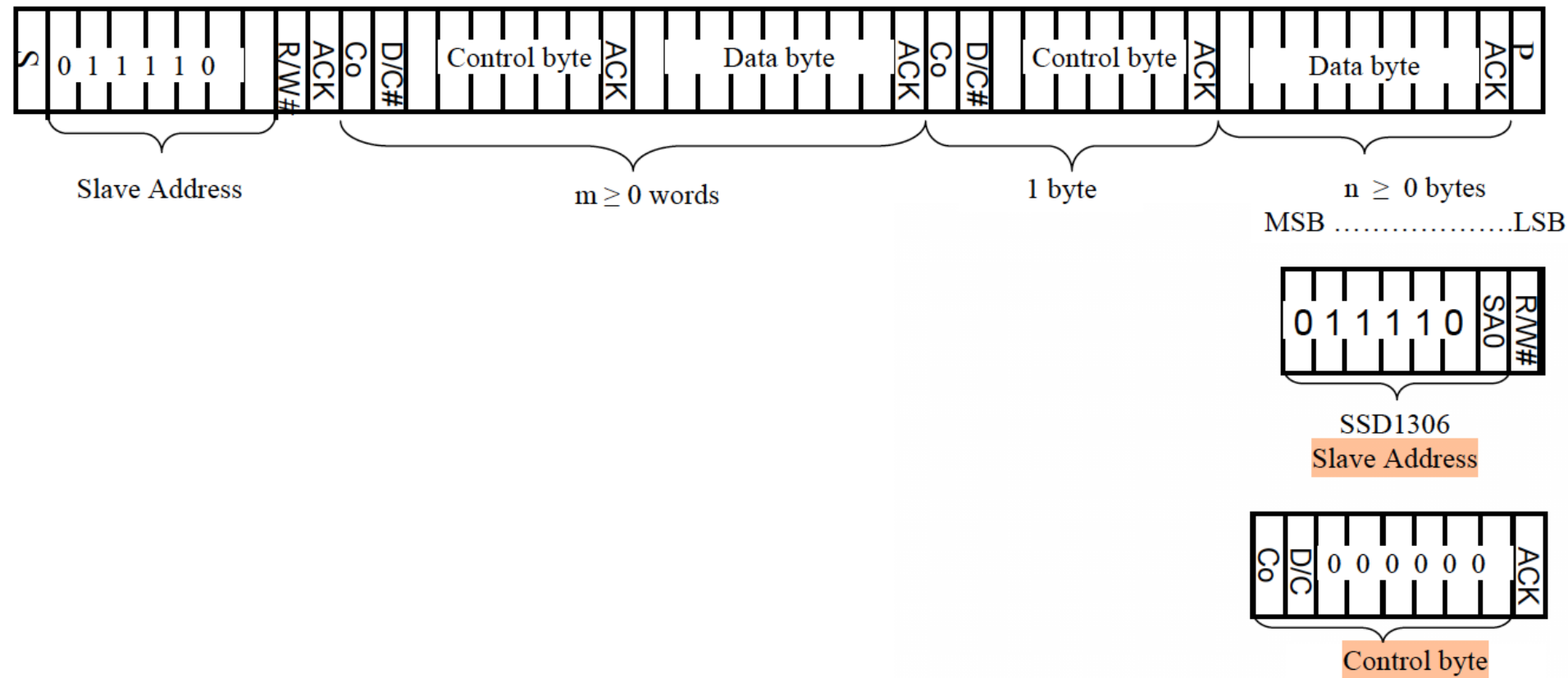
This tells SSD1306 that the next byte is a command

```
void sendCommand(uint8_t command) {  
    Wire.beginTransmission(SSD1306_I2C_ADDRESS);  
    Wire.write(0x00); // Co = 0, D/C# = 0 (Command) 0b0000-0000  
    Wire.write(command);  
    Wire.endTransmission();  
}
```


Note:

- Co – Continuation bit
- D/C# – Data / Command Selection bit
- ACK – Acknowledgement
- SA0 – Slave address bit
- R/W# – Read / Write Selection bit
- S – Start Condition / P – Stop Condition

Write mode



Control byte: 0b01000000
This tells SSD1306 that the next bytes are display data

```
void sendData(uint8_t data) {
    Wire.beginTransmission(SSD1306_I2C_ADDRESS);
    Wire.write(0x40); // Co = 0, D/C# = 1 (Data) 0b0100-0000
    Wire.write(data);
    Wire.endTransmission();
}
```

Page addressing mode (A[1:0]=10xb)

In page addressing mode, after the display RAM is read/written, the column address pointer is increased automatically by 1. If the column address pointer reaches column end address, the column address pointer is reset to column start address and page address pointer is not changed. Users have to set the new page and column addresses in order to access the next page RAM content. The sequence of movement of the PAGE and column address point for page addressing mode is shown in Figure 10-1.

Figure 10-1 : Address Pointer Movement of Page addressing mode

	COL0	COL 1		COL 126	COL 127
PAGE0					
PAGE1					
:	:	:	:	:	:
PAGE6					
PAGE7					

In normal display data RAM read or write and page addressing mode, the following steps are required to define the starting RAM access pointer location:

- Set the page start address of the target display location by command B0h to B7h.
- Set the lower start column address of pointer by command 00h~0Fh.
- Set the upper start column address of pointer by command 10h~1Fh.

Page addressing mode (A[1:0]=10xb)

In page addressing mode, after the display RAM is read/written, the column address pointer is increased automatically by 1. If the column address pointer reaches column end address, the column address pointer is reset to column start address and page address pointer is not changed. Users have to set the new page and column addresses in order to access the next page RAM content. The sequence of movement of the PAGE and column address point for page addressing mode is shown in Figure 10-1.

Figure 10-1 : Address Pointer Movement of Page addressing mode

	COL0	COL 1		COL 126	COL 127
PAGE0					
PAGE1					
:	:	:	:	:	:
PAGE6					
PAGE7					

In normal display data RAM read or write and page addressing mode, the following steps are required to define the starting RAM access pointer location:

- Set the page start address of the target display location by command B0h to B7h.
- Set the lower start column address of pointer by command 00h~0Fh.
- Set the upper start column address of pointer by command 10h~1Fh.

- 1): We can update the GDDRAM per page, page address from 0xb0 to 0xb7
- 2): We will define the start column address of both the lower and upper pointer: with lower to be 0x00, and upper to be 0x10
- 3): We can then update the page RAM with the buffer we created

- 1): We can update the GDDRAM per page, page address from 0xb0 to 0xb7
- 2): We will define the start column address of both the lower and upper pointer: with lower to be 0x00, and upper to be 0x10
- 3): We can then update the page RAM with the buffer we created

```
void updatePage(int i) {  
    sendCommand (0xB0 | i); // Set the page address (0xB0 to 0xB7)  
    sendCommand (0x00); // Set lower column address  
    sendCommand (0x10); // Set higher column address
```

```
Wire.beginTransmission(SSD1306_I2C_ADDRESS);  
Wire.write(0x40); // Co = 0, D/C# = 1 (Data)  
for (uint8_t col = 0; col < SCREEN_WIDTH; col++) {  
    Wire.write(buffer[i * SCREEN_WIDTH + col]);  
}  
Wire.endTransmission();  
}
```

```

#include <Arduino.h>
#include <Wire.h>

#define SSD1306_I2C_ADDRESS 0x3C // Change to 0x3D if necessary
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define BUFFER_SIZE (SCREEN_WIDTH * SCREEN_HEIGHT / 8)

uint8_t buffer[BUFFER_SIZE]; // Screen buffer

// Function to send a command to the SSD1306
void sendCommand(uint8_t command) {
    Wire.beginTransmission(SSD1306_I2C_ADDRESS);
    Wire.write(0x00); // Co = 0, D/C# = 0 (Command) 0b0000-0000
    Wire.write(command);
    Wire.endTransmission();
}

// Initialize the SSD1306 display
void initSSD1306() {
    sendCommand(0xAE); // Display OFF
    sendCommand(0xD5); // Set display clock divide ratio
    sendCommand(0x80); // Default clock divide ratio
    sendCommand(0xA8); // Set multiplex ratio
    sendCommand(0x3F); // 1/64 duty
    sendCommand(0xD3); // Set display offset
    sendCommand(0x00); // No offset
    sendCommand(0x40); // Set start line address
    sendCommand(0x8D); // Charge pump setting
    sendCommand(0x14); // Enable charge pump
    sendCommand(0xAF); // Display ON
}

// Clear the display buffer
void clearBuffer() {
    for (uint16_t i = 0; i < BUFFER_SIZE; i++) {
        buffer[i] = 0x00;
    }
}

```

```

void updatePage(int i) {
    sendCommand(0xB0 | i); // Set the page address (0xB0 to 0xB7)
    sendCommand(0x00); // Set lower column address
    sendCommand(0x10); // Set higher column address

```

```

Wire.beginTransmission(SSD1306_I2C_ADDRESS);
Wire.write(0x40); // Co = 0, D/C# = 1 (Data)
for (uint8_t col = 0; col < SCREEN_WIDTH; col++) {
    Wire.write(buffer[i * SCREEN_WIDTH + col]);
}
Wire.endTransmission();
}

```

```

// Update the display with the buffer contents
void updateDisplay() {
    for(int page=0; page<8; page++)
        updatePage(page);
}

```

```

// Draw a pixel in the buffer
void drawPixel(uint8_t x, uint8_t y) {
    uint16_t index = x + (y / 8) * SCREEN_WIDTH;
    buffer[index] |= (1 << (y % 8));
}

```

```

void setup() {
    Wire.begin(21, 22); // SDA, SCL pins
    initSSD1306();
    clearBuffer();
    updateDisplay(); // Send the buffer to the display
}

```

```

void loop() {
    drawPixel(11, 10);
    updateDisplay();
}

```

10 min: Complete the circuit if you haven't, test the code ;)

And now you know how to work and update the display with pixels, you will be able to create your own drawing library moving forward

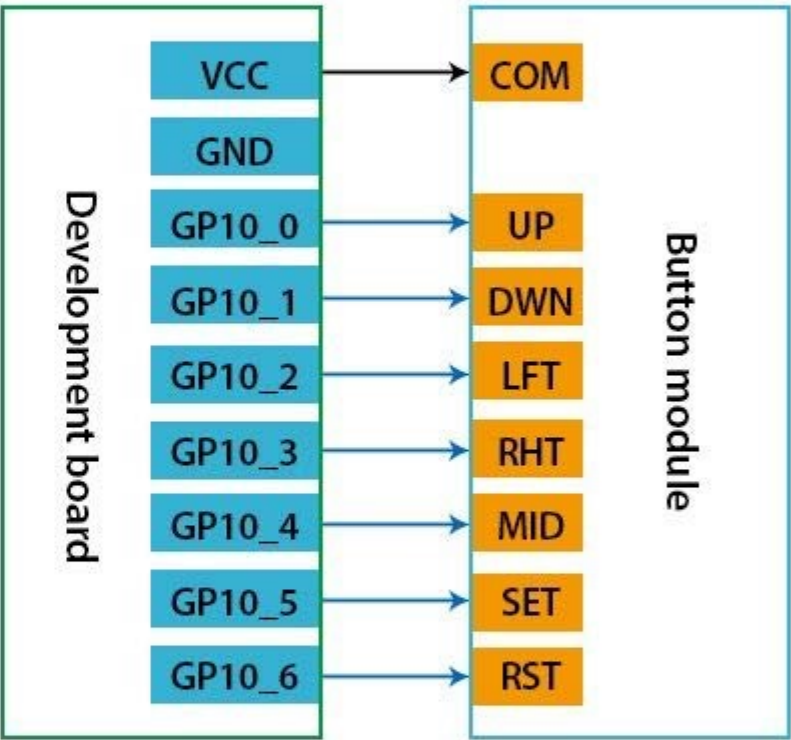
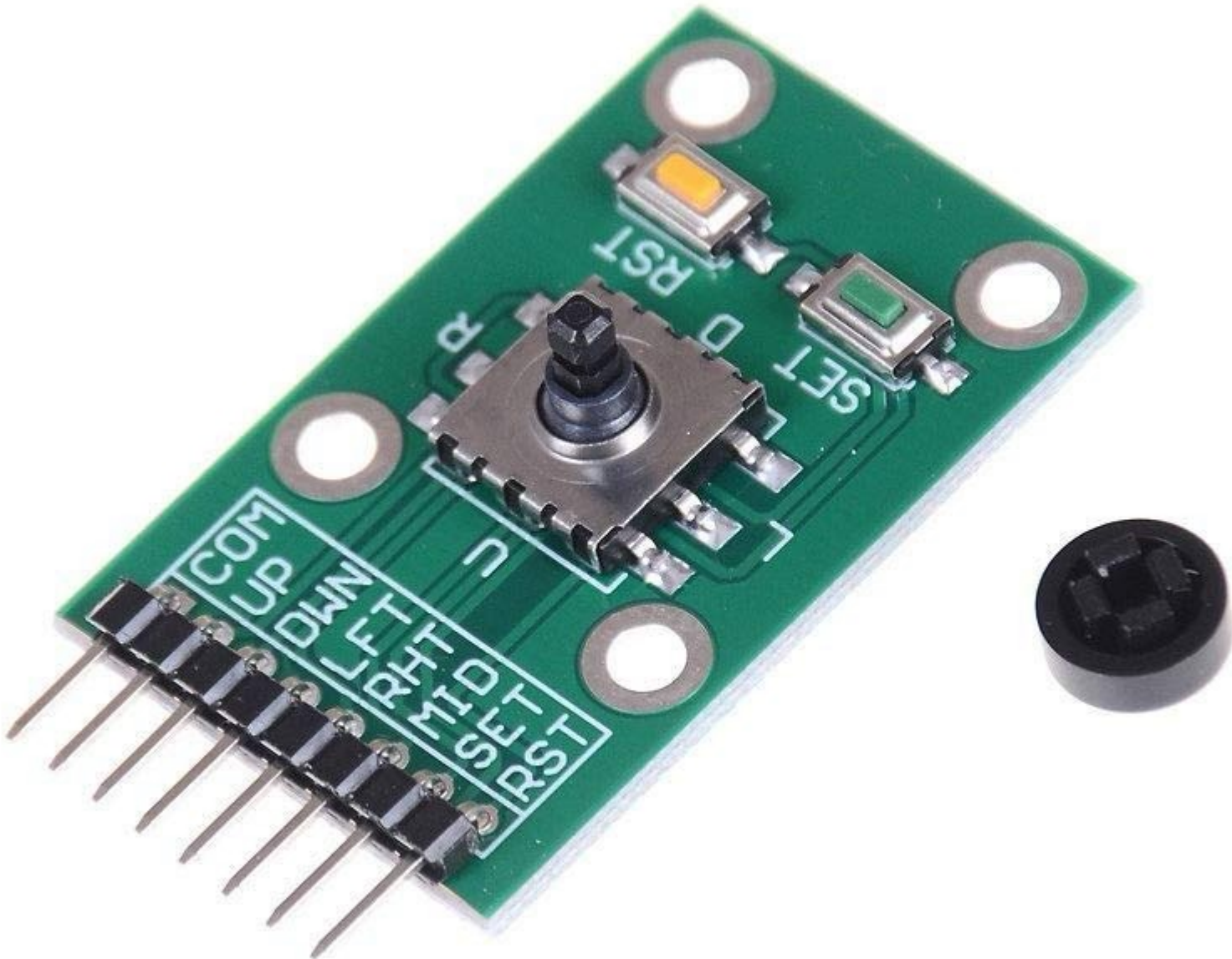
```
void drawRectangle(int x, int y, int width, int height) {  
  
}
```

```
void drawCircle(int x, int y, int r) {  
}
```

...

https://github.com/adafruit/Adafruit_SSD1306

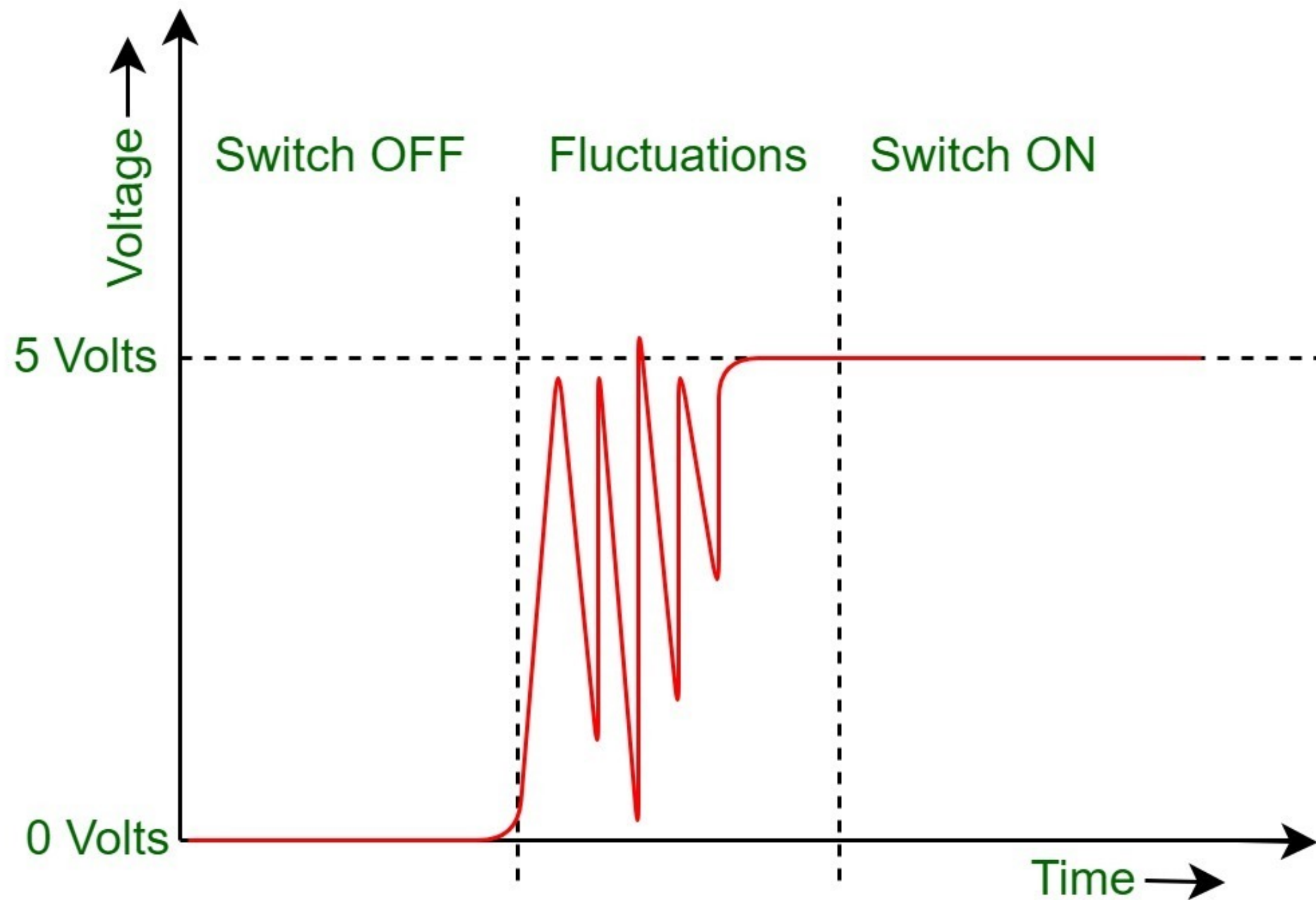
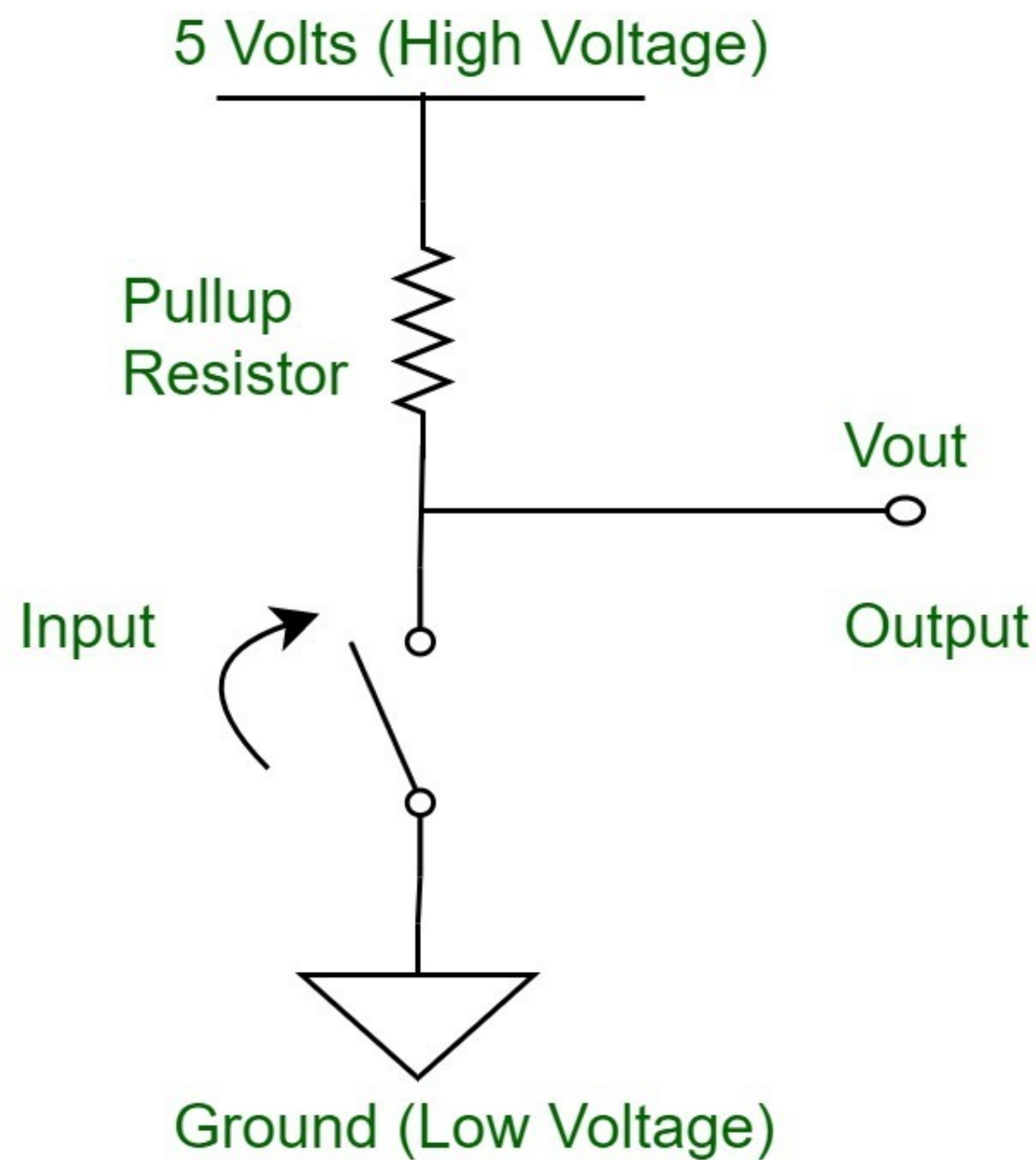
Five Direction Button Module



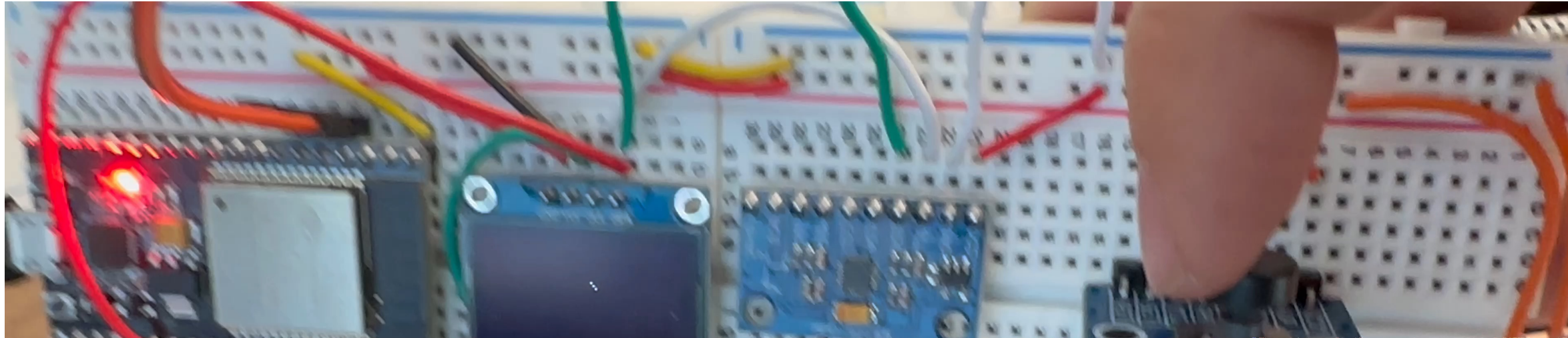
Any GPIOs

10 min: Write a test code to test the functionality of the 5-direction joystick

Button debouncing:



20 min: Write a simple game where you can use the joystick to control the movement of the pixel on the display



Assignment:

